

Homework #2

Due date: 12/11/2022

Notes:

- If you used Python codes for your questions, compress them along with an answer sheet (a docx or pdf file).
- Name your winzip file as “**CS41507_hw02_yourname.zip**”
- Attached are “**myntl.py**”, “**lfsr.py**”, and “**client.py**” that you can use for the homework questions.
- Use “**client.py**” to communicate with the server. The main server is located at the campus therefore you need to **connect to the campus network using VPN (ONLY IF YOU ARE OUTSIDE THE CAMPUS)**. Then, you can run your code as usual. See IT website for VPN connections.
- **The server’s IP address** is provided as “http://10.92.55.4:6000”. This IP is provided by default in **client.py**. Check the code.

1. (18 pts) Use the Python function **getQ1** in “**client.py**” given in the assignment package to communicate with the server. The server will send you a number **n** and the number **t**, which is the order of a subgroup of Z_n^* . Please read the comments in the Python code.

Consider the group Z_n^* .

- a. (4 pts) How many elements are there in the group? Send your answer to the server using the function *checkQ1a*.
- b. (8 pts) Find a generator in Z_n^* . Send your answer to the server using the function *checkQ1b*.
- c. (8 pts) Consider a subgroup of Z_n^* , whose order is **t**. Find a generator of this subgroup and send the generator to the server using function *checkQ1c*.

Solutions are in the CS411-507_HW2_Q1.py file

2. (10 pts) Use the Python code **getQ2** in “**client.py**” given in the assignment package to communicate with the server. The server will send 2 numbers: **e**, and **c**.

Also, **p** and **q** are given below where $n=p \times q$

p =
 12971142097853774608886730934213242678590198968987459448589637155501
 99865737054261727888057261785094677480406791687340958844335970176040
 12172054368990172572715857537355524013819947862920969421702067385445
 12224267306495899196866613854438036552045602995241496202871180617578
 4928131826127885820644091951344318387

q =
 17406667240508597265780888177897852058280976323514735837433240996632
 29872907454164052204143230047829067573625791571179144949271984426455
 81197584273451379119673753279114693557694861941678350357667191083878
 10082892019850377453927128926363364664736419813018030413809928153266
 0260760636194367337370132530987351081

Compute $m = c^d \bmod n$ (where $d = e^{-1} \bmod \phi(n)$). Decode m into Unicode string and send the text you found to the server using the function **checkQ2**.

Answers are in the form of: **b'In another parallel universe, your university ID would be: ' + secretNumber + ' . In case you do not like it, ask Loki to change it for you.'**

Solutions are in the CS411-507_HW2_Q2.py file

3. (18 pts) Consider the following attack scenario. You obtained the following ciphertexts that are encrypted using SALSA20 and want to obtain the plaintexts. Luckily, the owner of the messages is lazy and uses the same key and nonce for all the messages. You also know that the owner uses a 64-bit key.

Key: 14656892184006070584

ciphertext 1:

b"Vbq\x8a\xe3\xb7Rg1-\x14\x8bNS\xeb\x01\xbd\xdf\x1f\x14\x84{\xdanX,\xa5\x98RM\x98\r\xd7\x1e\x9d0\x14\xa7\x8cX\xcb\xad\xfb2\xc9\x1f\xc1j\xef\x908I\xe0\xcf\x10%.ulh\xe7\xd6\x9d<\xb9a\xda\xb0\xa2d\xe9\x18\xef9\x99ttP\x9blw\x0e\xe7\xd6\xbb1\xf4?\x16kf\x87\x19\xbe\x940\xe8\x1d\x08\xe4\xff)\x99'j\xda\x191=|H"

ciphertext 2:

b'\eda\x01q+j\x8c\x06[\xa2/\xb8\xcaX\x1f\x8f:\xc97\x0f)\xa5\x84Y\t\xdc\x07\xd2L\xb3V\x14\xad\x8bU\x99\xa3\xfb2\x9dK\xc8V\xab\xdd\nS\xe9\xcf\x05\$r,\t<\x9e\xd0\x9b<\xbcx\x99\xaf\xed7\xfb9\x13\xff9\x88r\\\x9b}>\x1d\xeb'

ciphertext 3:

b'ea,\x14\x88NW\xbfh\xb9\xcdX\x0f\x83}\xc0cX5\xa5\x9e\x1e^\xd0\x03\xc5\x1e\xa3U@\xa1\x85H\xc0'

However, during the transmission of the ciphertexts, some bytes of 2 out of the 3 messages were corrupted. One or more bytes of the nonce parts are missing.

Attack the ciphertexts and find the messages. (See the Python code **salsa.py** in Sucourse+)

(Hint: You can assume new Salsa instance is created for each encryption operation)

message1 = "In the Imaginary world of Witchcraft and Wizardry, You pick side either with Grindelwald or Dumbledore"

message2 = "This might be a wrong decryption of the message. It is up to you to decide"

message3 = "Do what is right not what is easy"

Solutions are in the CS411-507_HW2_Q3.py file

4. (12 pts) Solve the following equations of the form $ax \equiv b \pmod n$ and find all solutions for x if a solution exists. Explain the steps and the results.

a. $n = 1593089977489628213419978935847037520292814625191902216371975$

$a = 1085484459548069946264190994325065981547479490357385174198606$

$b = 953189746439821656094084356255725844528749341834716784445794$

$d = \gcd(a, n) = 1$. There is a solution.

Solution :

$778214478105812676636756719791275483076257993476674633986349$

b. $n = 1604381279648013370121337611949677864830039917668320704906912$

$a = 363513302982222769246854729203529628172715297372073676369299$

$b = 1306899432917281278335140993361301678049317527759257978568241$

$d = \gcd(a, n) = 7$ and 7 does not divide b . There is no solution.

c. $n = 591375382219300240363628802132113226233154663323164696317092$

$a = 1143601365013264416361441429727110867366620091483828932889862$

$b = 368444135753187037947211618249879699701466381631559610698826$

$d = \gcd(a, n) = 2$ and 2 divides b then there are two solutions.

One solution can be obtained as follows:

$\tilde{a} = \frac{a}{d} = 571800682506632208180720714863555433683310045741914466444931$

$\tilde{n} = \frac{n}{d} = 295687691109650120181814401066056613116577331661582348158546$

d divides b

$\tilde{b} = \frac{b}{d} = 184222067876593518973605809124939849850733190815779805349413$

Now we have an equation:

$\tilde{a} * \tilde{x} \equiv \tilde{b} \pmod{\tilde{n}}$

$\tilde{x} \equiv$

$214452804613963486637685100244716357445929679678527437334799 \ 51014049$
 $5723613606819499501310772970562507011340109785493345$

d. $n = 72223241701063812950018534557861370515090379790101401906496$
 $a = 798442746309714903219853299207137826650460450190001016593820$
 $b = 263077027284763417836483401088884721142505761791336585685868$

$d = \gcd(a, n) = 4$ and 4 divides b then there are two solutions.

One solution can be obtained as follows:

$$\tilde{a} = \frac{a}{d} = 199610686577428725804963324801784456662615112547500254148455$$

$$\tilde{n} = \frac{n}{d} = 18055810425265953237504633639465342628772594947525350476624$$

d divides b

$$\tilde{b} = \frac{b}{d} = 65769256821190854459120850272221180285626440447834146421467$$

Now we have an equation:

$$\begin{aligned} \tilde{a} * \tilde{x} &\equiv \tilde{b} \pmod{\tilde{n}} \\ \tilde{x} &\equiv \end{aligned}$$

{896049914695400631894506737412603426006253201283161992061189518603
 39961353869399140376877946054778848148808512468685 37007670765227307
 106903774016343288683551443096333862945309 5506348119049326034440840
 7655808631312324038043859213421933}

5. (10 pts) Consider the following binary connections polynomials for LFSR:

$$p_1(x) = x^6 + x^5 + x^4 + x + 1$$

$$p_2(x) = x^6 + x^2 + 1$$

$$p_3(x) = x^5 + x^3 + 1$$

Do they generate maximum period sequences? (**Hint:** You can use the functions in `lfsr.py`)

$p_1(x) = x^6 + x^5 + x^4 + x + 1$: when we start with any initial state and run the LFSR, we obtain a sequence of period 63, which is maximum. Then, the answer is YES.

$p_2(x) = x^6 + x^2 + 1$: when we start with 000001 and run the LFSR, we obtain the following sequence whose period is 14, which is less than the maximum period of 63. Then the answer is NO.

$p_3(x) = x^5 + x^3 + 1$: when we start with any initial state and run the LFSR, we obtain a sequence of period 31, which is maximum. Then, the answer is YES.

6. (12 pts) Consider the following sequences that are generated using different random number generators. Which of the sequences are predictable? (**Hint:** You can use the functions in `lfsr.py`)

$x_1 = [0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0]$

```
x2 = [0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0,
1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1]
```

```
x3 = [1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1,
1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0,
0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1]
```

x2 and x3 are predictable since the linear complexity of the sequences are less than half of the length of the sequences. x1 is unpredictable. If we apply the Berlekamp-Massey algorithm to them as follows

```
print ("L1 and C1(x): ", BM(x1), len(x1))
print ("L2 and C2(x): ", BM(x2), len(x2))
print ("L3 and C3(x): ", BM(x3), len(x3))
```

L1 and C1(x): (45, [1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1]) 89

[illegible][illegible]

7. (20 pts) Consider the following ciphertext bit stream encrypted using a stream cipher. And you strongly suspect that an LFRS is used to generate the key stream:

c_{text} =

[1, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1,
1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0,
1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
0, 1, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0,
1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1,
1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 1,
0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 0,

```

0, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1,
0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0,
1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1,
0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1,
1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1,
0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0,
1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0,
0, 0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1,
0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0,
1, 0, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1,
1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0,
0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0,
0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 1,
1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1,
0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0,
0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0,
1, 1, 0, 0, 1, 1, 1, 1, 1, 1]

```

Also, encrypted in the ciphertext you also know that there is a message to you from the instructor; and therefore, the message ends with the instructor's name. Try to find the connection polynomial and plaintext. Is it possible to find them? Explain if it is not.

Note that the ASCII encoding (seven bits for each ASCII character) is used.

(Hint: You can use the `ASCII2bin(msg)` and `bin2ASCII(msg)` functions (in `bonus_helper.py`) to make conversion between ASCII and binary)

Connection polynomial can be found as $x^{27} + x^8 + x^7 + x^1 + 1$

The plaintext is:

```

Dear Student,
You have worked hard, I know that; but it paid off:)
You have just earned 20 points.
Congrats!
Best, Atil Utku Ay

```